CS6600 | COMPUTER ARCHITECTURE

Unified Assignment

A Coherent Multicore system

From a five-stage in-order processor, to multi-cycle execution and non-blocking caches, to speculative TAGE-SC-L execution, to a directory-based shared-memory multicore connected by a 2D mesh.

This document combines the four supplied assignment specifications and the PA4 starter README, interface contracts, and counter definitions into one implementation-oriented reference.

Scope	What this guide consolidates
PA1	Five-stage IF/ID/EX/MEM/WB processor, hazard stalls, branch/jump behavior, HALT, Harvard memory, builder wiring.
PA2	Seven execution pipelines, in-order issue/retirement, multi-cycle timing, split L1/L2 caches, MSHRs, writebacks, replacement.
PA3	TAGE-SC-L predictor, BTB, speculative execution, checkpoints, squash/recovery, prediction and recovery metrics.
PA4	2D mesh NoC, private-cache/home-directory MSI, LLC integration, SC/TSO, DRAM scheduling, event counters.

HOW TO USE THIS DOCUMENT

Read it in order when reconstructing the system. The final PA4 starter headers and docs remain authoritative for exact C++ signatures, enum values, capacities, and timing. This is a consolidated study/implementation guide, not a replacement assignment issued by course staff.

Prepared from the files supplied in this conversation. Source names are listed in the final section.

Navigation and implementation map

Part	What you will find
1. System evolution	How PA1, PA2, PA3 and PA4 fit together; what is retained and what is superseded.
2. PA1 foundation	Five-stage pipeline, hazards, control flow, instruction format and structural wiring.
3. PA2 processor core	Seven pipelines, timing, issue/retirement, control and AGU behavior.
4. PA2 cache hierarchy	Harvard L1/L2 caches, MSHRs, writebacks, arbitration, replacement, and invariants.
5. PA3 prediction and speculation	TAGE-SC-L, BTB, prediction lifecycle, checkpoints, wrong-path squash and metrics.
6. Final PA4 system	Topology, tile mapping, data path, file ownership, and migration checklist.
7. PA4 Task 1	XY/YX routing, virtual channels/classes, arbitration, link timing and backpressure.
8. PA4 Task 2	MSI cache/directory protocol, messages, transaction flows, LLC, evictions and atomics.
9. PA4 Task 3	SC/TSO memory ordering, store buffer, forwarding, FENCE, XCHG and litmus test.
10. PA4 Task 4	FCFS/custom memory scheduling and DRAM bank/row behavior.
11. PA4 Task 5	Counter events, occupancy, stalls, accounting equations and independent observer.
12. Build and testing path	Exact commands, public tests, staged validation and debugging priorities.
13. Viva and reference	Transaction walkthrough checklist, glossary, source traceability and papers.

THE SINGLE MOST IMPORTANT DISTINCTION

PA1 is the conceptual baseline. PA2 replaces the one-cycle execution/memory model with seven multi-cycle pipelines and non-blocking caches. PA3 adds prediction/speculation to that PA2 machine. PA4 migrates the PA3 processor into a multicore framework and adds coherent shared data memory. Do not rebuild PA4 as the old PA1 five-stage design.

Reading precedence: which specification wins?

- PA1 explains the original five-stage pipeline; its one-cycle/no-prediction implementation is superseded by PA2.
- PA2 supplies the seven-pipeline, execution, cache, ownership and timing model retained by PA3, except where PA3 specifies speculation behavior.
- PA3 supplies the predictor and speculative pipeline behavior carried into PA4.
- PA4 Assignment 4 plus README.md, docs/INTERFACES.md, docs/COUNTERS.md and public headers govern final multicore interfaces, messages, cycle order, capacities and counters.

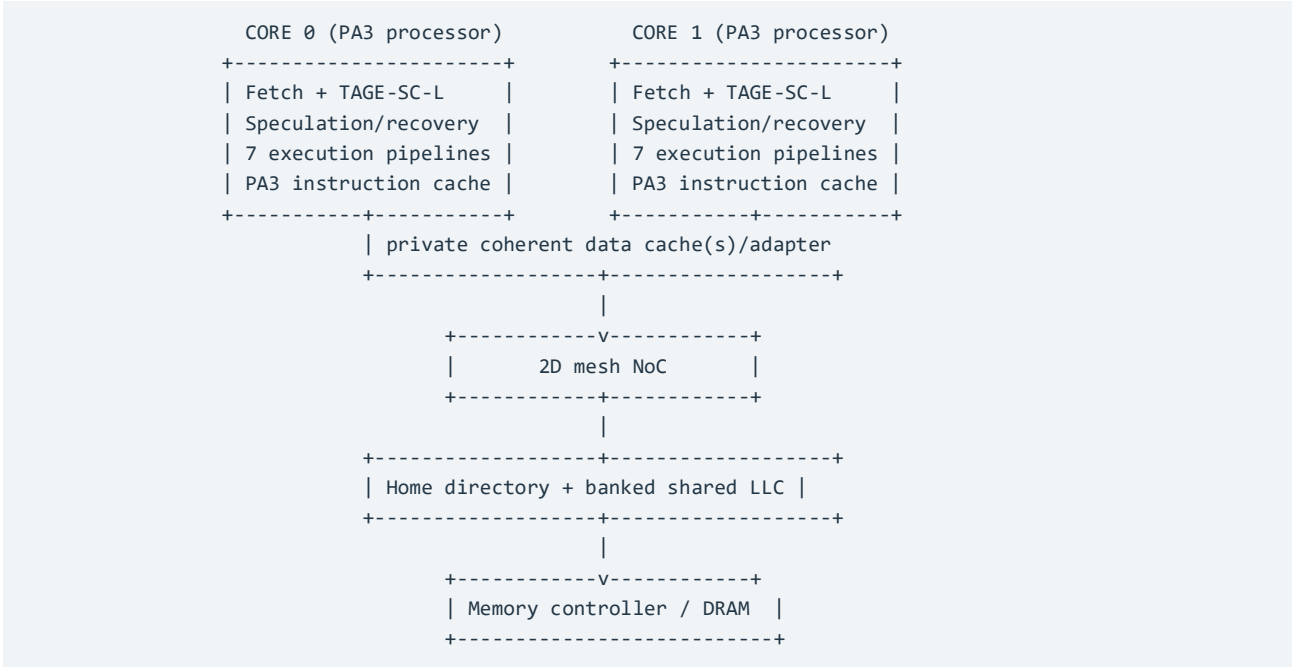
The A4 starter is authoritative for exact C++ declarations and configured capacities. This guide reorganizes the requirements but does not invent signatures or default values.

1. How the four assignments form one machine

The assignments are cumulative in concepts, but they are not four independent hardware blocks that should all be pasted together. Each assignment changes the processor organization. The final PA4 system uses the PA3 processor as each core, with PA4-specific adapters and a new coherent data path.

Stage	Architecture introduced	What carries forward
PA1	Single five-stage in-order pipeline: IF, ID, EX, MEM, WB. One-cycle stages; hazards stall; branches resolve in EX.	ISA and architectural behavior; the idea of in-order program execution; RAW hazard reasoning; control-flow and HALT semantics.
PA2	Seven independent execution pipelines; multi-cycle latency and initiation interval; completion queue and in-order retirement; split non-blocking caches.	One in-order issue point; RAW/WAW checks; branch/control semantics; instruction/data separation; architectural correctness.
PA3	TAGE-SC-L predictor, BTB, speculative fetch/dispatch, branch checkpoints, wrong-path squash and recovery.	PA2 execution latency, cache configuration, hazards, issue discipline and in-order retirement.
PA4	Multiple PA3 cores; 2D mesh; home directories; MSI coherent private data caches; shared banked LLC; SC/TSO; DRAM request scheduling and counters.	PA3 core/predictor/speculation; PA2 execution and instruction-cache hierarchy; PA4 adapter and all supplied framework contracts.

1.1 Final PA4 mental model



The default PA4 configuration is a 3 x 3 mesh with four core tiles, four home/LLC-bank tiles, and one memory-controller tile. Mesh dimensions remain configurable. Endpoint tile IDs are assigned row-major: cores first, homes next, controllers last; unused tiles are routers only.

2. PA1 foundation: the original in-order processor

PA1 asks for the simplest correct processor: a five-stage in-order pipeline whose stage movement and hazards match the simulator cycle by cycle. It establishes the architectural behavior that later assignments preserve while replacing the internal execution machinery.

2.1 Five stages and pipeline registers

Stage	Role
IF - Instruction Fetch	Read the instruction at the current PC and advance/hold the fetch state.
ID - Instruction Decode	Decode opcode, identify register uses, detect hazards and generate control information.
EX - Execute	Perform ALU work and resolve BEQ/BNE/J control-flow decisions.
MEM - Memory	Perform the data-memory action for loads/stores.
WB - Write Back	Commit register results; HALT becomes architecturally effective here.

The required pipeline state includes PC and the inter-stage registers IF/ID, ID/EX, EX/MEM and MEM/WB. The stages, pipeline registers, ALU, and control/hazard logic must be registered with ProcessorBuilder according to the supplied SPEC.md and builder API. Registered objects must correspond to the logic actually used; decorative registrations are insufficient.

2.2 Hazard behavior: stall rather than forward

- There is no forwarding path in PA1. If an instruction in ID needs a source register whose producer has not written it back, ID waits.
- On a register hazard, hold the PC and IF/ID state, insert a bubble into ID/EX, and allow older instructions past ID to continue moving.
- The required behavior is cycle-accurate: stage occupancy, stalls, bubbles, redirect timing and final state all matter.

2.3 Control flow and HALT

- BEQ, BNE and J are recognized in ID, but branch/jump resolution occurs in EX. Fetch waits until the actual next PC is known.
- HALT stops new fetches when encountered; the machine actually halts only when HALT reaches WB.
- Each stage takes exactly one cycle in PA1. This fixed-latency rule is replaced by PA2 execution-unit latency/II in later assignments.

2.4 Program format and memory model

```
INST <pc> <opcode> <rd> <rs1> <rs2> <immediate>
MEM  <data_address> <initial_word_value>

# Example: x3 = x1 + x2
INST 0x08 ADD 3 1 2 0

# Initialize data memory word at byte address 0x20
MEM 0x00000020 42
```

The machine is Harvard: instruction fetch and data loads/stores use separate memories. An instruction at PC 0x08 and a data word at address 0x08 do not alias. The text .pisa loader handles instruction records; students implement opcode-to-control interpretation, not machine-code encoding.

2.5 PA1 validation and rubric

- Run `make`, then `./pipesim example.pisa`; use the simulator commands `n`, `pipe`, `s`, `p`, `graph` and `q` for cycle stepping, stage snapshots, statistics, state and datapath inspection.
- Test RAW and load-use hazards, taken/not-taken branches, jumps and HALT. Compare final registers, memory and halt status, but also compare the cycle-by-cycle trace.
- PA1 grading emphasized final architectural state (40), exact cycle behavior (30), genuine builder structure (20), and ability to explain hazard and redirect logic (10).

3. PA2 processor core: seven multi-cycle execution pipelines

PA2 is a new starter skeleton, not a patch to PA1. Port the useful front-end/hazard reasoning, but implement the new execution and cache machinery using PA2 interfaces. PA2 replaces the original one-cycle EX/MEM/WB implementation with independent pipelines and a completion/retirement split.

3.1 Seven execution units and dispatch

Opcode(s)	Execution pipeline
ADD / ADDI	ADD
SUB	SUB
AND	AND
OR	OR
XOR	XOR
BEQ / BNE / J / HALT	CONTROL
LW / SW	AGU

- There is one in-order issue/dispatch point. At most one instruction dispatches per cycle, and a blocked ID instruction cannot be bypassed by a younger one.
- A dispatched instruction enters exactly one of the seven independent pipelines. Execution can finish out of order, but retirement remains in order and is limited to one instruction per cycle.
- There is no execution-result forwarding. A completed but unretired result is not an ID forwarding source.

3.2 Latency, initiation interval and current/next state

Each execution pipeline is stateful and configured by latency and initiation interval (II). It has exactly latency + 1 one-entry stages; latency zero still has one stage. The nominal result boundary is:

```
nominal_result_boundary = dispatch_boundary + latency + 1
```

A component reads boundary-N state, makes cycle-N decisions, plans boundary-(N+1) state, and changes current state only at global commit. `BeginCycle()` copies current to planned state; `OutputIntent()` is observational; `CanAccept()` is side-effect free; `Operate()` modifies planned state; `Commit()` publishes it; `Snapshot()` reports real state; `HasWork()` reports transient work.

- Move pipeline stages back-to-front. If stage N is blocked, stage N-1 cannot advance into it.
- A hold preserves the same operation ID, metadata, payload and age. A retry cannot repeat allocations, writes or accepted-event counters.
- A slot freed in cycle N is not reusable for admission until cycle N+1. A registered link can dequeue its old item and accept a new item in the same cycle; the consumer does not see the new item until after commit.

3.3 Correct in-order issue checks

The PA2 guide requires the ID instruction to pass checks in this order:

Order	Check	Required condition
1	Completion capacity	Reserve a bounded location for the future result.
2	RAW	No source reads a value still outstanding.
3	WAW	A younger write cannot overtake an older write to the same architectural destination.
4	Unit availability	Mapped execution pipeline can accept at the dispatch boundary.
5	Completion reservation	Reserve the required future fixed-result slot.

Dispatch is atomic: latch operands, reserve completion capacity and insert into one execution unit together. If any check fails, no dispatch-owned state changes.

3.4 Completion, retirement, CONTROL and AGU

- Execution completion is separate from architectural update. A younger operation may finish first, but the retirement queue waits for the oldest instruction. Retire at most one per cycle and pass its exact completion record once to the framework.
- HALT is architecturally effective only at retirement, not at decode or CONTROL dispatch.
- CONTROL handles BEQ, BNE, J and HALT. Recognition, fetch stalling, branch resolution and HALT retirement are separate events.
- AGU handles LW/SW. For LW: lhs = rs1 value, rhs = uint32(immediate), store_value = 0. For SW: lhs = rs1, rhs = uint32(immediate), store_value = rs2 value. Effective address is lhs + rhs modulo 2^32, but the resulting access must still be aligned, in range, non-wrapping and within one cache line.
- If the pending-data table or D-cache input link is full, the AGU output holds. It must not freeze unrelated execution pipelines.

4. PA2 cache hierarchy: what the PA3 core inherits

PA2 adds a split Harvard cache hierarchy. Instruction and data paths remain separate at both levels; there is no unified L2, and L1/L2 line sizes may differ.

```
Instruction path:  IF -> L1I -> L2I -> InstructionMemory
Data path:        AGU -> L1D -> L2D -> DataMemory
```

4.1 Cache pipeline and address mapping

- Each cache controller has exactly three pipeline stages: INDEX -> LOOKUP -> OUTPUT. With no holds, a request transferred in cycle N becomes current INDEX at boundary N+1, LOOKUP evaluates in cycle N+2, and a hit response first becomes an upward candidate at boundary N+3.
- For line size L and S sets, split the address into byte offset, set index and tag. Each controller uses its own configured line size and set count; never hard-code a global line size.
- Before mutation, validate nonzero size, no 32-bit address wrap, and that the complete access fits within one line of the receiving cache.
- LOOKUP compares all ways conceptually in parallel. A duplicate valid-tag match in two ways is an invariant violation. A read hit returns the requested bytes; an L1D write hit updates bytes and marks the line dirty.

4.2 Non-blocking misses and MSHRs

An MSHR represents one outstanding miss for one local aligned cache block plus a bounded list of waiting requests.

1. Search for an existing MSHR for the block. If found, append one waiter if capacity permits; do not create another lower read.
2. If no matching MSHR exists, require both a free MSHR and an eligible victim.
3. Allocate the lowest-index currently free MSHR and reserve the victim atomically.
4. Create one lower read for the complete local cache line.
5. If the waiter list is full, hold the new request rather than creating a duplicate MSHR.
6. When the full lower line arrives, match the exact MSHR, install the line, replay waiters in arrival order, and produce bounded responses.
7. Keep the filled way pinned until waiter responses drain. Never install a partial line.

4.3 Stores, writebacks and ordering

- The data hierarchy is write-back and write-allocate. Dirty eviction captures line bytes in a bounded writeback entry.
- Writebacks move UNSENT -> IN_FLIGHT -> ACK_READY -> freed. An IN_FLIGHT payload is immutable.
- UNSENT writebacks with overlapping byte ranges may merge if acknowledgement capacity permits. The oldest by (arrival_order, lower_id) survives; incoming bytes are applied last so they win overlapping bytes. ACK_READY is not a merge target.
- An overlapping lower demand read is blocked while a writeback is UNSENT or IN_FLIGHT. A non-overlapping request may proceed.

4.4 Bandwidth and replacement

Per-cache action	Maximum per cycle
Upper requests accepted	1
Lower responses accepted	1
Upper responses enqueued	1
Lower requests enqueued	1
Normal or fill LOOKUPs	1

- Upward response candidates are the ordinary OUTPUT response, oldest ready unsent waiter from each filled MSHR, and oldest unsent ACK waiter from each ACK_READY writeback. Choose minimum (arrival_order, response_id).
- Among eligible UNSENT demand MSHRs, choose minimum (primary_arrival_order, lower_id). Demand normally wins over writeback; writeback wins when its buffer has no current free entry or when the oldest eligible writeback has accumulated eight accepted demand-grant bypasses.

- Replacement options are Tree-PLRU and SRRIP. Controller logic owns invalid-way preference and eligibility; replacement policy selects only among eligible ways. `PeekVictim()` and `DebugState()` are observational; mutating operations act on planned state and publish only at commit.

Configuration is authoritative: execution latency must be nonnegative and II positive; line sizes are powers of two at least four bytes; set counts are positive powers of two; capacities are positive where required; Tree-PLRU ways must be compatible with its tree; memory latency and outstanding capacity are positive; derived sizes are legal; and each L2 line size is at least its corresponding L1 line size. Use configured values, not hard-coded conveniences.

4.5 PA2 validation priorities

- Test RAW/WAW, out-of-order completion with in-order retirement, delayed CONTROL, AGU backpressure, L1/L2 hits and misses, MSHR merging, ordered replay, dirty writeback, overlapping writeback merge, read-after-write ordering, and finite-buffer backpressure.
- Use the PA2 degenerate bring-up recommendation: latency-0 execution units and always-hit memory should reproduce the PA1 trace before enabling realistic multi-cycle execution and cache misses.

5. PA3: TAGE-SC-L prediction and speculative recovery

PA3 extends the PA2 processor with prediction during fetch and execution along a predicted path. It must recover from wrong predictions without changing the architectural result. PA4 migrates this PA3 core, rather than reverting to the non-speculative PA2 front end.

5.1 Integrated predictor organization

Implement one integrated TAGE-SC-L predictor in PredictorOne (slot 0). Slots 1 and 2 are fixed shadow predictors supplied by the starter and remain unchanged. The statistical corrector and loop predictor live inside PredictorOne.

Component	Required behavior
TAGE	Base predictor plus tagged tables with geometric history lengths; provider/alternate selection; prediction counters; allocation and replacement; usefulness management; history updates.
Statistical corrector (SC)	Signed-counter tables; combine table outputs; train under the chosen paper variant; decide whether to retain or override TAGE.
Loop predictor (L)	Identify loops; track iterations/trip counts; maintain confidence and replacement; select loop prediction when appropriate.
BTB / equivalent	Finite structure identifying branch PCs and retaining targets.

HARDWARE REALISM CONSTRAINT

Every table in the predictor (tagged TAGE tables, SC tables, loop table, and base lookup structure) must use direct indexing: hash the lookup context to one entry and compare its stored tag. A broad associative search is not an acceptable substitute. Aliasing is expected.

BEQ and BNE require direction prediction. J is unconditional. HALT is not a branch. Indirect branches, calls and returns are outside the stated ISA.

5.2 Predictor interface lifecycle

Method	When and why
Predict	During fetch, before decode. Uses PC and history to predict direction and target; the result/metadata must travel with the instruction.
ObserveInstruction	At decode, checks whether the earlier branch-identification guess was correct. It does not revise the prediction.
Update	When the branch executes and the real outcome is known; compares the saved prediction and trains predictor state and branch metrics.
Reset	Clears predictor tables and counters.
StorageBits	Reports logical storage in bits for every predictor table/counter.
Metrics	Returns counters maintained by Predict, ObserveInstruction and Update.

Retain the original prediction and metadata until branch resolution or squash. Document the history-update and recovery policy. Tables should be sized as a deliberate storage/performance trade-off, and the report must include tags, counters, targets, histories, confidence and replacement state.

5.3 Fetch, sequence numbers and checkpoints

- Use predictor slot 0 to choose next fetch address. Predicted taken selects its target; otherwise fetch PC+4.
- Carry the original prediction metadata with the instruction. Use dynamic sequence numbers to distinguish repeated executions of the same PC.
- Maintain branch checkpoints. Stall when checkpoint capacity is exhausted. A stall must not create repeated checkpoints or repeat an instruction observation callback.
- Younger arithmetic instructions may dispatch/execute speculatively, subject to PA2 hazards and resources. Younger loads, stores, branches, jumps and HALT wait until the older branch resolves. Dispatch remains in program order.

5.4 Resolution and recovery

Two resolution calls are distinct and must both be used:

- `BranchPredictionSystem::Resolve` compares the saved predictor guess with the actual outcome, drives learning and updates prediction/branch metrics. Its return value is for prediction accounting.
- `SpeculationController::Resolve` uses its own checkpoint and returns the authoritative RecoveryDecision that determines whether redirect/squash is required. It releases the resolving branch checkpoint itself.
- Do not call `RemoveYoungerCheckpoints` to release the branch that just resolved. That helper discards checkpoints strictly younger than the supplied sequence and is intended for wrong-path cleanup.
- On misprediction, redirect to the actual next PC. Handle false branch identification as well as direction and target errors.

Recovery preserves the resolving branch and all older instructions. Discard younger state from fetch, IF/ID, execution pipelines, completion storage, reservations and checkpoints. Invalidate outstanding wrong-path fetch responses so they cannot re-enter the pipeline. Release predictor metadata owned by discarded instructions and restore speculative history if it was updated early.

Preserve the simulator stage ordering WB, MEM, EX, ID, IF and its current/next-state discipline. A squash must not be undone by a later stage copying stale state forward. The supplied predictor is synchronous during fetch: do not add a predictor pipeline or predictor cycles. The branch CONTROL-stage latency is the branch-resolution latency.

ARCHITECTURAL SAFETY RULE

A squashed instruction must never write an architectural register or memory and must never terminate the program. Completed wrong-path operations are squashed just like unfinished ones. The PA3 restriction against speculative memory dispatch is what prevents a wrong-path store from becoming visible.

5.5 PA3 statistics and correctness checks

- Track cycles, retired instructions, speculative fetches/dispatches and recovery cycles; predictor lookups, branch-identification hits/misses, correct predictions, direction errors, target errors and false branch identifications; resolved branches, redirects, max speculation depth and discarded fetches/operations/completions.
- Count a target error separately from a direction error; do not count one branch twice. False branch identification is a separate category.
- Report IPC = retired instructions / cycles. Report MPKI = 1000 x (direction errors + target errors) / retired instructions.
- Also report provider usage for each TAGE component, SC override frequency and correctness, loop predictor selection/correctness, logical storage bits, and per-component table reads/writes.
- Compare final registers, architectural memory and termination against trusted execution. Use Machine::PrintState or equivalent behavior that overlays dirty cache contents and pending writebacks; do not inspect backing memory alone.
- Test direction errors, target errors, changing loop behavior, wrong-path completion, wrong-path HALT and fetch responses arriving after recovery. Consistent counters alone do not prove correctness.

6. PA4 final system: topology, migration and files

PA4 builds a configurable shared-memory multicore system using the PA3 processor. It adds the 2D mesh, distributed home directories, private coherent data caches, shared banked LLC, memory controllers, TSO option, custom DRAM scheduling and event counters.

6.1 Default topology and address mapping

Item	PA4 contract
Mesh	Default 3 x 3; dimensions remain configurable.
Endpoints	Four core tiles, four home/LLC-bank tiles, one memory-controller tile; remaining tiles are routers only.
Tile IDs	Row-major: tile ID = y * configuration.x + x. Endpoint types are assigned cores first, homes next, controllers last.
Router ports	Positive X, Negative X, Positive Y, Negative Y, Local.
Line size	Coherent cache line is 64 bytes. `line_address` is 64-byte aligned.
Address units	CPU addresses are aligned 32-bit word byte addresses. Word index inside a line = (address % 64) / 4.
Home mapping	line_index = line_address / 64; home selected by line_index modulo number of homes.
Controller mapping	Controller selected by line_index modulo number of controllers; endpoint/vector offsets are defined in the starter.

The LLC and private cache storage in PA4 are finite direct-mapped structures. Directory metadata survives LLC data eviction: the LLC is non-inclusive. If a private cache owns a line in M, the LLC/backing memory may contain stale data.

6.2 Data path and separation from PA3 data caches

```
Core -> private coherent data cache -> mesh -> home directory + shared LLC
     -> memory controller -> DRAM

Instruction path: migrated PA3 instruction-cache hierarchy remains active.
Data path:        PA4 coherent private cache + home/LLC/DRAM system.
```

One coherent data cache access per core may be active at a time. A home directory permits one active transaction per line. Requests that conflict with an active transaction are queued; the queue capacity is router_depth. A core must continue servicing coherence messages while its local memory request is stalled.

6.3 PA3 migration into the PA4 starter

The PA4 README gives the migration map. Copy completed PA3 implementations into these matching locations:

PA3 source component	PA4 destination
Core and top-level design	design.cc; design/core.{cc,h}; design/control.{cc,h}
Execution pipeline	execution_unit.cc; execution_unit/impl.cc; execution_unit/internal.h
Cache implementation state and replacement	cache/cache_controller/internal.h; cache/replacement.cc
Speculation	speculation.cc; speculation/internal.h
Predictor	predictors/predictor_one.{cc,h} and custom predictor helpers

DO NOT OVERWRITE THESE A4 FILES

Retain A4 pipesim.h, framework/*, framework_api.cc, design/instruction_helpers.cc and execution_unit/helpers.cc. Keep the supplied shadow predictors unchanged. Merge custom shared-header fields without deleting A4 interfaces. Keep submission3 and earlier folders unchanged.

6.4 Required PA4 core merges

8. In Core::Memory, call Machine::MemorySequenceMayIssue(sequence) before AGU handoff. If it returns false, stall without consuming the execution output.
9. In Core::Decode, treat XCHG displacement as rhs and rs2 as store_value. FENCE has no register operands. Supplied helpers select AGU and hazard behavior.
10. AGU preserves instruction/prediction metadata, computes lhs + rhs, and carries store_value. memory_write is true only for SW; the adapter distinguishes FENCE and XCHG by opcode.
11. FENCE and XCHG wait behind unresolved branches like LW and SW.

Migration check: the arithmetic workload must report register 3 = 12 on cores 0 and 1. Run it before implementing the PA4 algorithms.

6.5 PA4 functions you implement

File	Function(s)	Responsibility
noc/routing.cc	select_output_port; select_router_input	XY/YX routing and round-robin/custom output arbitration.
coherence/cache.cc	accept_cache_request; handle_cache_message	Private-cache MSI request handling and protocol message handling.
coherence/directory.cc	handle_directory_message; advance_directory_transaction	Home-directory message handling, transaction phases and LLC integration.
memory/ordering.cc	decide_memory_action	TSO store buffering, forwarding, bypass and fence issue decisions.
memory/scheduling.cc	select_memory_request	FCFS and custom eligible-request selection.
statistics.cc	count_event; sample_occupancy_and_stalls	Accepted-event counters and cycle-start occupancy/stall samples.

PA4 component	Marks
Task 1 - 2D mesh routing and arbitration	20
Task 2 - distributed MSI coherence	40
Task 3 - memory ordering (SC/TSO)	15
Task 4 - memory scheduling	10
Task 5 - event counters	10
Code clarity and integration	5

You may add helpers and transient-state fields, but preserve public interfaces, capacities, address mappings, configuration, and documented timing. Do not modify the supplied LLC or DRAM models, framework files, or the independent observer.

7. PA4 Task 1: 2D mesh routing and arbitration (20 marks)

Every remote communication must traverse the mesh. Do not directly deliver a remote message from source to destination. A single routing order is selected globally for each run.

7.1 XY and YX routing

Mode	Route construction
XY	First move in X until destination column is reached, then move in Y.
YX	First move in Y until destination row is reached, then move in X.

`RoutingContext.current_router` is the router making the decision, not the packet source. `destination_router` is the final destination. Return Local only when the current router is the destination. X/Y coordinates increase in the positive directions. Tile ID is y * mesh_width + x.

```
dx = destination_x - current_x
dy = destination_y - current_y

# XY: if dx != 0, choose +/-X; otherwise choose +/-Y.
# YX: if dy != 0, choose +/-Y; otherwise choose +/-X.
# At destination, choose Local.
```

7.2 Packet classes, channels and input IDs

Class / channel	Messages	Purpose
Requests = 0	GetS, GetM, PutS, PutM, MemRead, MemWrite	Acquisitions/releases and LLC-memory requests.
Snoops = 1	RecallS, RecallI, Inv	Forwarded recall and invalidation traffic.
Replies = 2	Ack, DataS, DataM, GrantAck, PutAck, MemData, MemAck	Responses, data and acknowledgements.

Each router has five physical ports: +X, -X, +Y, -Y and Local. There are three separate input-buffer classes per port, for 15 input IDs:

```
input_id = port * 3 + channel(message_type)   // 0 through 14
```

`ArbitrationCandidate.input` is the input ID, not its vector position. `select_router_input` returns one such ID or nullopt when the candidate list is empty. Candidates already passed readiness, routing, storage and link checks. If candidates are nonempty, select one; do not idle.

7.3 Arbitration

- Round-robin compares eligible input IDs cyclically starting from round_robin_cursor modulo 15. The framework updates the cursor after each grant.
- The custom policy is your choice, selected by `arbitration custom`. Define deterministic tie-breaking and avoid depending on host-language iteration order.
- Respect connected ports, output bandwidth, finite source/destination buffers and backpressure. Request, snoop and reply buffers are separate; response traffic must not be trapped behind blocked acquisitions.

7.4 Store-and-forward timing and packet sizes

- Before launching, reserve downstream storage. A physical link is shared by buffer classes.
- Link occupancy = ceil(packet_bytes / link_bytes). Arrival boundary = launch boundary + link occupancy + link_latency. Router latency applies before the next forwarding decision.
- Headers reserve 16 bytes. PutM, Ack, DataS, DataM, MemWrite and MemData reserve an additional 64 bytes. Even an invalidation Ack reserves a data-sized payload in this baseline.
- Source/destination capacity is per class; link occupancy is shared. A downstream-blocked cycle is counted once per router per cycle when any ready routed input is blocked by destination storage while its output is otherwise available.

7.5 Task 1 validation

- Use `pa4-noc` and `workloads/network.traffic`. Check exact routes, hop counts and unloaded equal-packet latency at increasing distances.
- Compare XY and YX and both arbiters on identical traffic. Include contention and full-buffer tests. With equal link timings, greater unloaded hop distance must produce greater latency.

8. PA4 Task 2: distributed MSI coherence (40 marks)

This is the central PA4 protocol. Implement both the private-cache controller and the home-directory controller as a single distributed state machine. Each private cache stores a local copy of a line; the directory at the line home serializes permission changes.

8.1 Stable MSI states and invariants

State	Meaning	Allowed local operation
I - Invalid	No valid readable/writable copy. At the directory, unowned/no private permission.	A load/store/XCHG must acquire permission.
S - Shared	Readable copy; may coexist with other S copies.	Load can complete. Store/XCHG requires M permission.
M - Modified	One core has exclusive writable ownership; its data is newest.	Load/store/XCHG can complete locally, subject to the CPU ordering layer.

MSI SAFETY INVARIANT

For one line, M is exclusive: no other core may simultaneously hold S or M. Equivalently, no writable copy may coexist with another readable copy of the same line. A directory grant must wait for every required invalidation/recall acknowledgement.

The directory tracks stable/transient state, owner, sharers and active transaction. Private caches track their line state and pending access. Directory metadata survives LLC data eviction, because the LLC is non-inclusive.

8.2 Protocol messages and their meanings

Message	Direction	Meaning
GetS	Core -> home	Request shared/read permission.
GetM	Core -> home	Request exclusive/modified permission, including an S-to-M upgrade.
PutS	Core -> home	Release a clean victim.
PutM	Core -> home	Release a modified victim and provide its current full-line data.
RecallS	Home -> current owner	Return latest modified data and downgrade owner to S.
RecallI	Home -> current owner	Return latest modified data and invalidate owner.
Inv	Home -> sharer	Invalidate a shared copy.
Ack	Core -> home	Acknowledge invalidation or recall; recall Ack carries current line data.
DataS	Home -> requester	Full line and shared permission.
DataM	Home -> requester	Full line and exclusive permission.
GrantAck	Requester -> home	Confirms line installed and pending access applied.
PutAck	Home -> evicting core	Eviction release acknowledged; victim can be freed.
MemRead / MemWrite	LLC -> memory controller	Read fill / full-line writeback.
MemData / MemAck	Controller -> LLC	DRAM response.

Message class: requests channel 0; snoops channel 1; replies channel 2. Packet IDs and coherence transaction IDs are different. `Packet.id` is assigned by send and identifies one transport packet. `MemoryRequest.id` is unique for the entire run and is used as the cache acquisition/eviction transaction ID. Preserve line_address, requester and transaction_id in protocol replies. Requester is the original core even when another core supplies recall data.

The `message(type, source_tile, destination_tile, line_address, transaction_id, data, requester)` helper constructs a packet but does not send it. Source and destination are tile IDs. `can_send(source_tile, type, count)` checks next injection capacity for that message class but does not reserve it; `send(packet)` may still return false. Preflight every message required to consume an input, including the combined count for multiple messages in the same class, and check every send result.

8.3 GetS: read miss

12. The private cache accepts a local read miss, records one active cache access and sends GetS to the line home using the request transaction ID.
13. The home consumes GetS. If the line already has an active transaction, queue the conflict (queue capacity = router_depth); otherwise start one directory transaction.
14. If a different core owns the line in M, the home must recall that owner before using LLC data. Send RecallS. The owner supplies the latest full line in Ack and downgrades to S.
15. If no private M owner holds newer data, obtain the line through the home LLC interface. Do not read backing DRAM directly from the directory.
16. The home sends DataS with the full line. The requester installs S, applies its pending load, and only then sends GrantAck.
17. The home consumes the matching GrantAck, completes the active transaction, and updates directory owner/sharer metadata.

8.4 GetM: write miss or S-to-M upgrade

18. The requester sends GetM. If it already has S, this is an upgrade; the requester retains the line data while seeking exclusive permission.
19. The home identifies all other sharers and/or the current M owner. It sends Inv to shared readers and RecallI to a modified owner as required.
20. Wait for all required acknowledgements. A recall acknowledgement carries the owner's latest data; the home must preserve it and update the LLC as needed before granting to the new owner.
21. Only after required acknowledgements and data acquisition can the home send DataM to the requester.
22. The requester installs/retains the line in M, applies the pending store or exchange, and then sends GrantAck.
23. The home completes the transaction only after matching GrantAck. It records the requester as the sole owner and clears other sharers.

An invalidation received during an upgrade removes the old readable copy but must preserve the outstanding acquisition. Grant write permission only after all required acknowledgements have been accepted.

8.5 Clean/dirty eviction and races

- A clean S victim is released with PutS. A dirty M victim is released with PutM carrying its full 64-byte line data. Retain dirty victim data until PutAck.
- The directory must preserve dirty data; LLC/backing memory can be stale while a core owns M.
- A recall may arrive while a dirty PutM is pending. The cache must still be able to return retained victim data.
- A delayed/queued stale PutM must never overwrite newer line data after ownership has changed. Match line, requester and transaction ID; validate current ownership before applying victim data.
- The protocol must service coherence messages while a core is stalled on its local memory request. Each core accepts at most one incoming packet and one local cache request per cycle, in that order.

8.6 Atomic exchange (XCHG)

XCHG obtains exclusive M permission and atomically reads the old aligned 32-bit word and writes the supplied new word. The old value is returned to the destination register. No competing core may interleave a conflicting operation between the read and write. XCHG requiring M permission is counted as an atomic miss, including when the local state is S; do not also count it as an ordinary store upgrade.

```
INST pc XCHG destination base source displacement

address   = register[base] + displacement
new_word  = register[source]
old_word  = memory[address]
memory[address] = new_word
register[destination] = old_word
```

Only aligned 32-bit accesses are supported. FENCE and XCHG are also handled by the PA4 memory-ordering layer; the coherence controller must not bypass that ordering policy.

8.7 LLC integration

- Use `llc_request(home_tile, LLCRequest)`. `write=false` requests a line. `write=true` supplies a complete updated line, marks it dirty and installs it even on an LLC miss; it is not write-through to DRAM.
- A successful enqueue returns true once. False means retry without changing the directory phase. The framework stamps the accepted time; do not invent it.

- Use `llc_reply(home_tile, line_address, transaction_id, requester)` only when the transaction is ready to consume a matching reply visible in current state. Advance phases so requests/replies cannot be consumed twice.
- Recall dirty owner data before relying on LLC data. The LLC may be stale under M ownership. Do not bypass the LLC by directly reading backing memory.

8.8 Exact callback and cycle rules

Rule	Required behavior
State model	`begin()` copies current to next. `evaluate()` samples counters, moves network, handles ordering, endpoints, LLC and DRAM; `commit()` publishes next and advances cycle.
Endpoint state	Cache/directory callbacks inspect and update next, not current. A later callback in the same cycle must see earlier planned invalidations.
Input consumption	Callback false means input not consumed: leave next, counters, queues and outgoing messages unchanged. True consumes exactly once.
Resource preflight	Check all required capacities before accepting input or mutating state. `can_send` checks next injection capacity but does not reserve it; check send result.
Directory advancement	At most one incoming packet per home per cycle. Advance entries that existed in current; a newly inserted entry begins advancing next cycle.
Snoop bandwidth	At most one new invalidation per home per cycle. Supplied can_send/send also limit new home snoops to one per cycle.
Progress	If a required resource is unavailable, keep the transaction in its current phase. Do not consume a reply before ready to use it.

Before a callback consumes an input, preflight every outgoing message needed. For multiple messages in one class, check the combined count. Space for a reply does not imply space for a request, and vice versa. `send(packet)` must be checked because capacity may be full.

Before calling `cache_complete(core_id, result_value)`, set `next.caches[core_id].pending`. Loads return the read word, stores return 0, exchanges return the old word. Fences are completed by the supplied ordering machinery. Do not call `cpu_complete` from a cache/directory controller.

9. PA4 Task 3: SC and TSO memory ordering (15 marks)

PA4 retains Sequential Consistency and adds Total Store Order. Select the model through configuration; TSO is not an instruction. The framework supplies the SC path and store-buffer storage; the student implements the TSO decision logic in `decide_memory_action`.

9.1 Sequential Consistency (SC)

Under SC, memory accesses appear in one global order that preserves each core's program order. The starter provides SC ordering. Do not accidentally weaken it while adding TSO.

9.2 TSO rules

- Each core buffers non-speculative stores in a bounded FIFO. Stores drain in FIFO order.
- A load forwards from the youngest older buffered store to the same word. If there is no same-word match, it may execute before older stores to different words become visible.
- Preserve load order. Stall a new store if the store buffer is full.
- FENCE waits for all older memory accesses and an empty store buffer before allowing younger memory operations.
- XCHG has the same ordering requirement and performs an indivisible exchange.
- Halted cores continue servicing coherence. Terminate only after buffered stores and outstanding work have drained.

9.3 Decision contract

Decision	Meaning
Wait	Keep CPU request; drain oldest buffered store if present.
BufferStore	Enqueue a store and acknowledge it to the core, only if capacity remains.
Forward	Complete a load with the value from the youngest older same-word store.
Issue	Send a load/store/exchange to cache; never issue a fence through this action.
CompleteFence	Complete the fence only when the store buffer is empty and older required work is complete.

`decide_memory_action` is called only in TSO mode and only when no earlier cache request is in flight. Its arguments are read-only; the framework moves queues and completes requests according to the returned decision. The FIFO contains only older stores, and the supplied in-order request interface preserves load order.

- When no CPU request is waiting, an admitted store waits at least four cycles before idle draining. Returning Wait can trigger draining immediately.
- Waiting fences and atomics continue draining older buffered stores. FENCE does not flush caches to DRAM.

9.4 Required store-buffer litmus test

```
Initial state: x = 0, y = 0; x and y are on different cache lines.

Core 0:                 Core 1:
  x <- 1                  y <- 1
  r0 <- y                 r1 <- x
```

Model / case	Can r0 = 0 and r1 = 0?
SC, no fences	No. Both zero is forbidden.
TSO, no fences	Yes, it may occur if both stores are still buffered when the loads execute.
TSO, FENCE between store and load on both cores	No. Both zero is forbidden.

A TSO run may produce other valid outcomes; observing two zeros is not required on every run. Additional required checks: same-word forwarding returns the core's latest buffered value; a different-word load can proceed while an older store remains buffered; a full buffer stalls the next store; FENCE blocks younger memory accesses until older work and the buffer drain.

10. PA4 Task 4: memory-controller scheduling (10 marks)

The DRAM model is supplied. Implement request selection only; preserve all timing/dependency behavior and do not modify the DRAM model.

10.1 FCFS

`eligible_requests` contains indices into `controller.queue`, not positions inside the eligible vector. Candidates target idle banks; they may still need PRE/ACT and timing waits, which the supplied DRAM engine enforces. Return an eligible index or no value if none are eligible. When the candidate list is nonempty, select one.

- FCFS selects the request with the earliest `Packet.controller_arrival`.
- Break ties by queue index. `Packet.injected` is the NoC injection cycle and is not the FCFS arrival timestamp.

10.2 Custom scheduling policy

Implement one policy of your choice under `scheduler custom`, selected by configuration. Define deterministic tie-breaking and a finite aging/starvation-prevention rule. One reasonable optional design is FR-FCFS: prefer a ready row hit, but force an aged request to the front after a finite wait threshold. That is a design suggestion, not a required policy name.

10.3 Address-to-bank/row mapping and evaluation

```
line_index = line_address / 64
bank = (line_index / controllers) % banks
row  = (line_index / controllers) / banks / 16
```

Compare FCFS and the custom policy on identical configurations and traffic patterns: same-row, alternating-row, independent-bank, and mixed read/write. Measure completion cycles, request latency, row hits/misses/conflicts, and maximum per-core waiting time. The educational DRAM model includes rows, ACT/PRE/RD/WR, shared burst bandwidth, turnaround/recovery and refresh; it is not a full DDR conformance model.

11. PA4 Task 5: counters and observability (10 marks)

Counters are part of correctness, not just performance reporting. They must describe accepted work exactly once, including network transfers, cache/coherence activity, store buffering, memory requests, DRAM row behavior, occupancy, stalls and latency.

11.1 Counting contract

- `count_event(system, component, event, amount)` stores a counter under `component.event`. Add amount for ordinary events; for names ending in `_max`, retain the maximum. Missing counts default to zero; reset clears all counters.
- Count accepted events once. A retry is not a new event. Sample occupancy at cycle start, before movement. Latency and wait are cycles; byte counts are bytes; occupancy is queued entries.
- Do not edit the independent observer, read/copy its totals, or duplicate framework-emitted accepted events from your controllers.

11.2 Events supplied by the framework

Prefix	Event family / definition
noc.T	packets_injected and injected_MESSAGE at successful source send; packets_delivered/delivered_MESSAGE at admission to destination router inbox; link_hops at completed inter-router transfer; link_bytes per launched hop; per-port grants; arbitration_losses; downstream_blocked_cycles; delivered_latency_sum/max.
cache.C	accesses_completed when cache_complete returns a result; excludes forwarded loads and fences.
order.C	stores_enqueued/stores_drained; loads_forwarded; loads_bypassed; fences_started/fences_completed.
llc.H	requests_accepted/completed; hits/misses per accepted lookup; dirty_evictions when a dirty victim MemWrite successfully sends.
mc.M	requests_accepted, requests_service_started, requests_completed, completed_latency_sum, read_bytes/write_bytes, and per-core queue wait sums/maxima.
mc.M / bank.B	row_hits, row_misses, row_conflicts classified at service start; ACT/PRE/RD/WR command counts when issued.

11.3 Events emitted by cache/directory controllers

Prefix	Event(s)	Definition
cache.C	loads_accepted, stores_accepted, atomics_accepted	One local request accepted exactly once.
cache.C	load_permission_hits, store_permission_hits, atomic_permission_hits	Matching readable permission for load; M permission for store/exchange.
cache.C	read_misses, write_misses, atomic_misses, upgrades	Accepted request lacks permission; S-to-M store is upgrade. XCHG needing M counts atomic_miss, not store upgrade.
cache.C	invalidations_received/applied	Inv consumed; applied when a readable or retained victim copy is invalidated.
cache.C	acks_sent	Ack for Inv/recall successfully sent; excludes GrantAck/PutAck.
cache.C	dirty_evictions, writebacks_sent	One each for every successfully sent PutM.
directory.H	requests_accepted	Home consumes GetS/GetM/PutS/PutM, including requests queued.
directory.H	transactions_started/completed	A request becomes active; completes at grant GrantAck or release PutAck.
directory.H	recalls_sent, invalidations_sent, acks_received	Count successful sends and matched/consumed acknowledgements.
directory.H	ownership_grants	DataM successfully sent, not received.
directory.H	conflicts_queued, completed_latency_sum	Queued conflicting requests; completion boundary minus transaction start.

11.4 Occupancy and stall samples

At every evaluated cycle, including idle cycles, collect queue_occupancy_sum, queue_occupancy_max and samples for each component queue prefix. Sample current state, excluding reservations where specified:

- Each `noc.T.queue.Q`: one router input queue length, excluding reservations; each `noc.T`: sum of its 15 input queue lengths.
- Each `mc.M`: controller request queue length, excluding bank-owned work. Each `llc.H`: LLC request queue length, excluding active work/replies.
- Each `order.C`: store buffer length, including a store being drained until its reply is consumed.

For `order.C`, record at most once per cycle: `buffer_full_cycles` when a CPU store waits and buffer is full; `fence_wait_cycles` when a CPU fence waits with nonempty buffer or in-flight cache request; `ordering_stall_cycles` when a CPU request waits on an earlier cache request or a fence/exchange waits for buffered stores. Categories may overlap; do not sum them as a unique total.

11.5 Accounting identities

Resource	During execution	At full drain
Network packets	injected = delivered + in-network	injected = delivered
Directory transactions	started = completed + active	started = completed
Buffered stores	enqueued = drained + buffered	enqueued = drained
Memory requests	accepted = completed + outstanding	accepted = completed
DRAM row classification	row hits + misses + conflicts = service starts	same identity

In-network packets include source queues, router inputs and link transfers; destination inbox packets already count as delivered. MC outstanding work includes queued requests and bank-owned work. Sum only component totals, not both component and per-port/per-bank copies. Directory start/completion events are independently observed by the framework.